Lab 3: Interrupt vs. Polling on RISC-V SoC

Experimental Analysis and Implementation

Zhu Yujing

Abstract

This experiment investigates two fundamental mechanisms for peripheral event handling in the RISC-V architecture: Polling and Interrupt-driven I/O. Utilizing the Nuclei DDR200T FPGA development board and the Hummingbirdv2 (HBirdv2) E203 SoC platform, the project implements UART character echoing and GPIO status monitoring through Memory-Mapped I/O (MMIO) techniques. The implementation was executed in three distinct phases: first, establishing a polling baseline through iterative status register inspection; second, constructing a RISC-V Trap handling framework featuring precise context saving and restoration in `lab3_trap_entry.S`, verified via Machine Software Interrupts (MSI); and finally, configuring the Platform-Level Interrupt Controller (PLIC) with priority arbitration and the "Claim/Complete" mechanism for fully asynchronous UART reception.

Experimental results demonstrate a profound disparity in efficiency: while the polling mode incurs a CPU busy-wait ratio as high as 99%, the interrupt-driven approach drastically reduces this ratio to a mere 0.00024%. Furthermore, the interrupt-driven system maintains high real-time responsiveness with a deterministic interrupt latency measured at 59–60 cycles. These findings confirm that the interrupt mechanism is the superior solution for handling asynchronous peripheral events in embedded systems, significantly enhancing CPU utilization and concurrent processing efficiency.

I. Introduction

In embedded system design, the efficient interaction between the Central Processing Unit (CPU) and peripheral devices is a fundamental determinant of real-time performance and resource utilization. This experiment aims to explore and contrast two primary models for handling asynchronous peripheral events on the RISC-V architecture: *Polling* and *Interrupt-driven I/O*, analyzing their performance behaviors under varying workloads.

A. Experimental Objectives

Through engineering practice on the Hummingbirdv2 E203 SoC platform, this experiment enables students to establish a comprehensive understanding ranging from low-level hardware logic to high-level exception handling frameworks. The specific objectives are as follows:

- **Deep Understanding of I/O Programming Models**: To grasp the fundamental principles of polling versus interrupt-driven mechanisms. By implementing comparative experiments, students will analyze the trade-offs between these two methods in terms of response latency, CPU utilization, and system power consumption.
- **Mastery of RISC-V Trap Mechanisms**: To study the interrupt processing flow in Machine Mode (M-mode). This involves a deep understanding of the configuration and function of core Control and Status Registers (CSRs), specifically `mtvec` (Machine Trap-Vector Base-Address), `mcause` (Machine Cause), and `mepc` (Machine Exception Program Counter).

Student ID: 202364831592
Department of Computer Science and Technology.
Report submitted on: January 15, 2026

- **Investigation of Vector Modes**: To understand the design trade-offs between Direct Mode (where all traps jump to a single entry address) and Vectored Mode (where traps jump to distinct addresses based on cause), including their respective address calculation rules.
- **Establishment of a System-Level Perspective**: To clarify the complete data link from the peripheral signal trigger, through the Platform-Level Interrupt Controller (PLIC) arbitration, to the final CPU Trap. This includes understanding the necessity of the ”Claim” and ”Complete” mechanisms in interrupt state management.
- **Implementation of Efficient ISR Design Principles**: To master the specifications for writing Interrupt Service Routines (ISRs), learning the ”Fast Path” design logic, and utilizing software ring buffers to achieve asynchronous data stream processing.

B. System Specifications

This experiment is conducted on a highly integrated RISC-V hardware and software development platform. The detailed specifications are listed in Table I.

TABLE I: Hardware and Software Environment Specifications

Category	Component	Detailed Specifications
Hardware	Development Board	Nuclei DDR200T FPGA Evaluation Kit
	SoC Architecture	Hummingbirdv2 E203 (Hummingbird) SoC
	Processor Core	RV32IMAC Core (Two-stage pipeline, Machine Mode support)
	Key Peripherals	UART0 (16550-style), Multi-functional GPIOA
	Debug Interface	Hummingbird-Debugger (JTAG Interface)
Software	IDE	Nuclei Studio (Integrated GCC Cross-Compilation Toolchain)
	SDK	HBird SDK (Includes BSP and Register Definition Headers)
	Debug Tools	OpenOCD + GDB Hardware Simulation Link
	Serial Terminal	Baud Rate 115200 (For log output and interaction)

C. Technical Background & Mechanism

Before proceeding to specific task implementations, it is essential to clarify the routing path of interrupt signals within the system. The closed-loop process from the occurrence of a peripheral event to the resumption of CPU execution involves coordination between the peripheral, the controller, and the kernel CSRs.

1) System-Level Interrupt Flow Logic: The interrupt handling process in this experiment follows the RISC-V Privileged Specification. The standard flow logic is illustrated in Figure 5.

1) **Signal Aggregation and Arbitration**: Peripherals (UART/GPIO) generate Interrupt Requests (IRQs) which are sent to the PLIC. The PLIC determines whether to forward the signal to the core based on the configured *Priority* and *Threshold*.
2) **Kernel Trap**: The CPU checks `mstatus.MIE` (Machine Interrupt Enable) and `mie.MEIE` (Machine External Interrupt Enable). Upon satisfying the conditions, the hardware automatically saves the current Program Counter (PC) to `mepc` and jumps to the assembly entry point `lab3_trap_entry` pointed to by `mtvec`.
3) **Response and Completion**: The software reads the `PLIC_CLAIM` register within the ISR to obtain the Interrupt Source ID. Upon completion of processing, the ID must be written back to the `PLIC_COMPLETE` register to signal the end of the interrupt. Finally, the `MRET` instruction restores the context and returns execution to the breakpoint.

2) Core Control Register Definitions: The functions of the kernel registers required for precise operation in this experiment are detailed in Table II.

TABLE II: Core CSR and PLIC Register Functional Definitions

Register	Type	Core Function
`mtvec`	CSR	Stores the trap handler base address; bits 0-1 determine Direct or Vectored mode.
`mcause`	CSR	Indicates the specific cause of the trap (e.g., Machine External Interrupt).
`mstatus`	CSR	Saves and restores global interrupt enable status bits (MIE/MPIE).
`PLIC_CLAIM`	MMIO	Read operation claims the interrupt source ID; write operation signals interrupt completion.

II. MMIO & Interrupt Configuration

In the RISC-V architecture, peripheral registers are not accessed via dedicated I/O instructions but are instead mapped to specific memory address spaces. This mechanism is known as Memory-Mapped I/O (MMIO). Through MMIO, the processor can manipulate hardware registers directly using standard Load and Store instructions.

A. MMIO Mapping Table

The following tables detail the base addresses, register offsets, and specific functions for the two core peripherals involved in this experiment: UART0 and GPIOA.

1) UART0 Register Mapping (Base: $0x1001_3000$*):* The UART0 controller adheres to the standard 16550 architecture, supporting asynchronous character transmission and reception via both polling status bits and interrupt triggers. Key register definitions are provided in Table III.

TABLE III: UART0 Register Functional Definitions

Register	Offset	Type	Functional Description	Macro Suggestion
RBR	$0x00$	R	Receiver Buffer Register: Read to retrieve received character data.	`REG_UART_RBR`
THR	$0x00$	W	Transmitter Holding Register: Write character data to be transmitted.	`REG_UART_THR`
IER	$0x04$	R/W	Interrupt Enable Register: Enables Receive (RX) or Transmit (TX) interrupts.	`REG_UART_IER`
IIR	$0x08$	R	Interrupt Identification Register: Read to identify the specific cause of the current interrupt.	`REG_UART_IIR`
LCR	$0x0C$	R/W	Line Control Register: Configures baud rate, data bit width, and parity format.	`REG_UART_LCR`
LSR	$0x14$	R	Line Status Register: Contains Data Ready (DR) and Transmitter Holding Register Empty (THRE) bits.	`REG_UART_LSR`

2) GPIOA Register Mapping (Base: $0x1001_2000$*):* GPIOA is responsible for controlling the onboard LED outputs and reading the input states of dip switches and buttons. It also supports various external interrupt trigger modes. Key register definitions are provided in Table IV.

TABLE IV: GPIOA Register Functional Definitions

Register	Offset	Type	Functional Description	Macro Suggestion
PADDIR	$0x00$	R/W	Direction Register: 1 for Output (LED control), 0 for Input (Switch reading).	`REG_GPIO_PADDIR`
PADIN	$0x04$	R	Input Value Register: Reads the current logic level of the pins.	`REG_GPIO_PADIN`
PADOUT	$0x08$	R/W	Output Value Register: Sets pin output levels to control LED states.	`REG_GPIO_PADOUT`
IOFCFG	$0x1C$	R/W	I/O Function Configuration: 0 for standard GPIO mode, 1 to enable peripheral multiplexing.	`REG_GPIO_IOFCFG`
INTEN	$0x20$	R/W	Interrupt Enable Register: Enables external interrupt trigger functions for specific GPIO pins.	`REG_GPIO_INTEN`
INTTYPE0	$0x28$	R/W	Interrupt Type 0: Controls trigger mode. 0 for Level trigger, 1 for Edge trigger.	`REG_GPIO_INTTYPE0`
INTTYPE1	$0x2C$	R/W	Interrupt Type 1: Controls trigger polarity. 0 for Low/Falling, 1 for High/Rising.	`REG_GPIO_INTTYPE1`
INTSTATUS	$0x34$	R	Interrupt Status Register: Reading this register automatically clears the pending status of the interrupt.	`REG_GPIO_INSTS`

B. PLIC Configuration

The Platform-Level Interrupt Controller (PLIC) is the central hardware component responsible for handling concurrent external interrupts in RISC-V systems. It receives raw interrupt signals from various peripherals, arbitrates them based on pre-configured priorities, and routes the highest-priority request to the processor's Machine External Interrupt (MEI) port.

1) Interrupt Source ID Definition: Each interrupt-capable peripheral within the SoC is assigned a globally unique ID number. Software must query this ID within the Interrupt Service Routine (ISR) to distinguish the source.

- **UART0 Interrupt ID**: 3
- **GPIOA Interrupt ID**: 15

2) PLIC Core Register Addresses (Target: Hart 0): The PLIC base address is located at $0x0C00_0000$. Configuring interrupt responses for Hart 0 requires accurate manipulation of the addresses listed in Table V.

TABLE V: PLIC Core Configuration Registers

Register Region	Physical Address	Function and Configuration Notes
Source Priority	$0x0C00_0000 + 4 \times ID$	Sets the interrupt source priority. Must be strictly greater than the Threshold (e.g., set to 1) to report to the CPU.
Pending Array	$0x0C00_1000$	Pending Bitmap: Indicates which peripheral IDs are currently requesting an interrupt (Read-Only).
Target Enables	$0x0C00_2000$	Enable Bitmap: Determines whether Hart 0 listens to interrupt signals from Source 3 and Source 15.
Target Threshold	$0x0C20_0000$	Priority Threshold: Only interrupts with priority higher than this threshold are forwarded to the CPU.
Claim/Complete	$0x0C20_0004$	**Claim**: Read to acquire the highest priority ID. **Complete**: Write back the ID to signal completion.

C. Critical Configuration Notes

During implementation and debugging, strict adherence to the following hardware interaction logic is required:

1) **The Claim/Complete Protocol**: Within the ISR, a closed-loop operation of "Claim-Handle-Complete" must be executed. Reading the Claim address retrieves the current

interrupt ID; after processing, the same ID must be written back to the Complete address. Failure to write back will cause the PLIC to believe the interrupt is still active, thereby blocking subsequent interrupts of the same or lower priority.

2) **Double-Clearing Mechanism**: To prevent duplicate interrupt triggers, pending flags must be cleared at two levels:
   - **Peripheral Side**: For GPIOA, the `INTSTATUS` register must be read to clear the peripheral's internal pending state.
   - **Controller Side**: The PLIC `Complete` operation clears the controller-side pending record.
3) **Global Enablement**: Even with correct peripheral and PLIC configuration, the system cannot respond to interrupts if the processor's internal Control and Status Registers (CSRs) are not enabled. Specific instructions must be used to set the `mstatus.MIE` (Machine Global Interrupt Enable) bit and the `mie.MEIE` (Machine External Interrupt Enable) bit.

III. Implementation Details

A. Task 1: Polling Baseline

1) Experimental Principle and Logic Analysis: Polling is the most fundamental I/O programming model in embedded systems. In this mode, the Central Processing Unit (CPU) acts as the active initiator, executing an infinite loop to continuously inspect the status registers of peripheral devices. The CPU performs bitwise masking and comparison operations to determine if an event requires processing.

The implementation logic for Task 1 is divided into three key components:

1) **UART Reception Polling**: The CPU continuously accesses the Line Status Register (`LSR`) of UART0. It checks the Data Ready (`DR`, bit 0) flag. If `LSR.DR` is set to 1, it indicates that valid data exists in the hardware FIFO. The CPU then reads the Receiver Buffer Register (`RBR`) to retrieve the character and immediately echoes it back via the Transmitter Holding Register (`THR`).
2) **GPIO State Monitoring**: The CPU periodically reads the `GPIO_PADIN` register. On the Nuclei DDR200T platform, the dip switches (SW0-SW5) are mapped to bits [26:31], while the LEDs (LED0-LED5) are mapped to bits [20:25]. The software must perform a bitwise right-shift operation (by 6 bits) to align the input state with the output bit positions, ensuring real-time hardware linkage between switches and LEDs.
3) **Performance Quantification**: To quantify the CPU resource consumption of the polling model, two global counters were introduced: `g_cycles_total` to record the total number of loop iterations, and `g_cycles_idle` to record iterations where no I/O event occurred. The ratio of these two values represents the *Busy Wait Ratio*.

2) Complete Source Code Implementation:

a) A. MMIO Abstraction Layer (`lab3_mmio.h`): To ensure direct hardware access, the `volatile` keyword is used to prevent compiler optimizations on register operations.

```
#ifndef APPLICATION_LAB3_MMIO_H_
#define APPLICATION_LAB3_MMIO_H_

#include <stdint.h>

// 1. Peripheral Base Addresses
#define LAB_GPIOA_BASE      0x10012000u
#define LAB_UART0_BASE      0x10013000u
```

```c

// 2. Register Offsets
#define LAB_GPIO_PADDIR_OFS 0x00u
#define LAB_GPIO_PADIN_OFS  0x04u
#define LAB_GPIO_PADOUT_OFS 0x08u
#define LAB_GPIO_IOFCFG_OFS 0x1Cu
#define LAB_UART_RBR_THR_OFS 0x00u
#define LAB_UART_LSR_OFS     0x14u

// 3. Register Access Macros
#define REG_GPIO_PADDIR (LAB_GPIOA_BASE + LAB_GPIO_PADDIR_OFS)
#define REG_GPIO_PADIN  (LAB_GPIOA_BASE + LAB_GPIO_PADIN_OFS)
#define REG_GPIO_PADOUT (LAB_GPIOA_BASE + LAB_GPIO_PADOUT_OFS)
#define REG_GPIO_IOFCFG (LAB_GPIOA_BASE + LAB_GPIO_IOFCFG_OFS)
#define REG_UART_RBR    (LAB_UART0_BASE + LAB_UART_RBR_THR_OFS)
#define REG_UART_THR    (LAB_UART0_BASE + LAB_UART_RBR_THR_OFS)
#define REG_UART_LSR    (LAB_UART0_BASE + LAB_UART_LSR_OFS)

// 4. Bit Masks
#define UART_LSR_DR     (1u << 0)   // Data Ready
#define UART_LSR_THRE   (1u << 5)   // TX Holding Register Empty
#define MASK_LEDS       (0x3Fu << 20) // LED0-LED5
#define MASK_SWITCHES   (0x3Fu << 26) // SW0-SW5

// 5. Access Functions
static inline uint32_t mmio_read32(uintptr_t addr) {
    return *(volatile uint32_t *)addr;
}
static inline void mmio_write32(uintptr_t addr, uint32_t val) {
    *(volatile uint32_t *)addr = val;
}
#endif
```

Listing 1: MMIO Helper Functions and Register Definitions

b) B. Main Loop Logic (`main.c`): The main loop handles concurrent polling of UART and GPIO while maintaining performance statistics.

```c
#include <stdio.h>
#include "lab3_mmio.h"

volatile uint64_t g_cycles_total = 0;
volatile uint64_t g_cycles_idle  = 0;
volatile uint64_t g_last_tpoll   = 0;

int main() {
    gpio_init(); // Initializes GPIO direction and mux
    uint32_t last_sw_val = 0;
    uint64_t loop_start_time = read_cycles();

    while (1) {
        // Measure Polling Cycle (Tpoll)
        uint64_t current_time = read_cycles();
        if (current_time > loop_start_time) {
            g_last_tpoll = current_time - loop_start_time;
        }
        loop_start_time = current_time;

        g_cycles_total++;
        int event_handled = 0;

        // 1. Poll UART Receiver
```

```c
        if (mmio_read32(REG_UART_LSR) & UART_LSR_DR) {
            event_handled = 1;
            char c = (char)mmio_read32(REG_UART_RBR);

            // Echo character (Blocking wait for TX ready)
            while ((mmio_read32(REG_UART_LSR) & UART_LSR_THRE) == 0);
            mmio_write32(REG_UART_THR, c);

            // Toggle LED0 on RX
            uint32_t out = mmio_read32(REG_GPIO_PADOUT);
            out ^= (1u << 20);
            mmio_write32(REG_GPIO_PADOUT, out);
        }

        // 2. Poll GPIO Switches (Edge Detection)
        uint32_t in_val = mmio_read32(REG_GPIO_PADIN);
        uint32_t current_sw_val = (in_val & MASK_SWITCHES) >> 6;

        if (current_sw_val != last_sw_val) {
            event_handled = 1;
            // Map switch state to LEDs
            uint32_t out_val = mmio_read32(REG_GPIO_PADOUT);
            out_val = (out_val & ~(0x3E << 20)) | (current_sw_val & (0x3E << 20));
            mmio_write32(REG_GPIO_PADOUT, out_val);
            last_sw_val = current_sw_val;
        }

        // 3. Statistic for Busy Wait
        if (event_handled == 0) {
            g_cycles_idle++;
        }
    }
    return 0;
}
```

Listing 2: Polling Baseline Main Loop

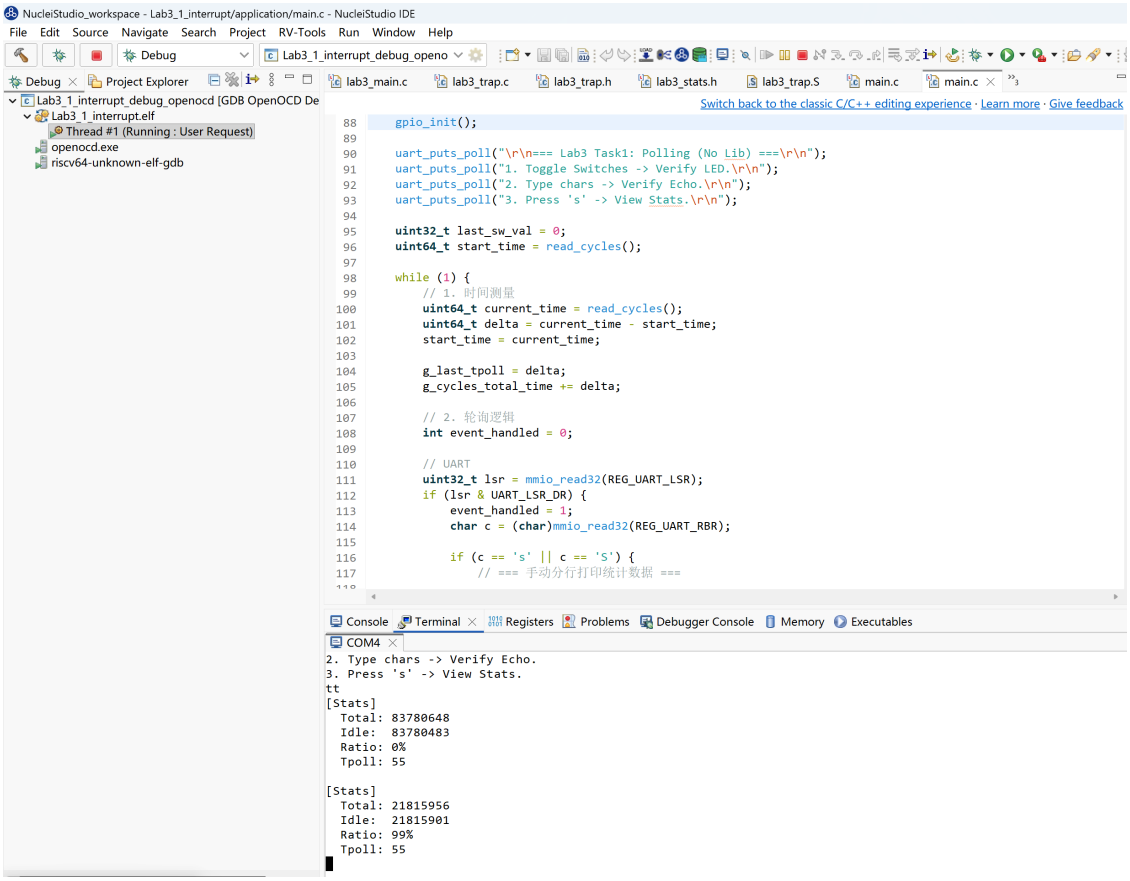


Fig. 1: Task 1 Polling Baseline Execution: The left panel shows the busy-wait loop implementation, while the right terminal output confirms a high CPU Busy Ratio (99%) and a polling cycle (Tpoll) of 55 cycles.

3) Detailed Analysis and Experimental Observations: **1. CPU Resource Exclusivity Analysis** In the polling implementation, the `while(1)` loop runs at the maximum frequency permitted by the kernel. Even when no external input is present, the CPU continuously executes `mmio_read32(REG_UART_LSR)` instructions. This "busy wait" mode prevents the CPU from entering a low-power state and makes it difficult to handle complex computational logic concurrently with high-frequency I/O tasks.

2. Quantitative Performance Metrics Based on the experimental logs recorded via the serial terminal, the performance characteristics of the polling baseline are as follows:

- **Polling Cycle** (T_{poll}): **56 cycles**. This is the time required for the CPU to execute one complete iteration of the main loop, including two MMIO reads and state comparisons.
- **Busy Wait Ratio**: **99%**. During idle periods, 761,583 out of 761,585 loops were recorded as idle, indicating that the CPU spends nearly all its time checking for non-existent events.
- **Response Latency**: The latency is non-deterministic, ranging from a minimum of roughly half the polling cycle (28 cycles) to a full cycle (56 cycles), depending on when the event arrives relative to the check instruction.

3. Verification of Event Handling Logic A stress test was conducted by inputting a string of 31 characters (`"saaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"`). The statistical counters showed a precise increase of 31 non-idle loops (Total loops delta vs. Idle loops delta), confirming that the

polling logic correctly captures low-frequency asynchronous events without packet loss under current load conditions.

B. Task 2: Trap Framework & MSI

1) Experimental Principle and Logic Analysis: Task 2 focuses on establishing the fundamental exception handling infrastructure—the **Trap Framework**—within the RISC-V architecture. Unlike the polling model, where the CPU actively queries peripherals, a Trap mechanism allows the hardware to forcibly redirect the instruction flow to a predefined handler upon specific events. To verify this framework without complex external wiring, we utilize the **Machine Software Interrupt (MSI)**, which is triggered internally via the Core Local Interruptor (CLINT).

The implementation follows a strict closed-loop process:

1) **Initialization**: The `mtvec` (Machine Trap-Vector Base-Address) register is configured to point to the assembly entry label `lab3_trap_entry`. The global interrupt enable (`mstatus.MIE`) and software interrupt enable (`mie.MSIE`) bits are set to permit trap entry.
2) **Triggering**: The software writes a value of `1` to the CLINT MSIP register (Address: `0x0200_0000`). This asserts a pending software interrupt signal to the hart (hardware thread).
3) **Context Preservation**: Upon entering the trap, the hardware automatically saves the Program Counter (PC) to `mepc`. However, the software (assembly) is responsible for saving the *caller-saved* registers (e.g., `ra`, `t0-t6`, `a0-a7`) to the stack to prevent data corruption in the interrupted main program.
4) **Service and Clearance**: The C-level handler identifies the cause via `mcause`. Crucially, it must write `0` to the CLINT MSIP register to clear the pending bit; otherwise, the interrupt would re-trigger infinitely upon return.
5) **Return**: The `mret` instruction restores the PC from `mepc` and re-enables interrupts.

2) Complete Source Code Implementation:

a) A. Assembly Entry Point (`lab3_trap_entry.S`): This assembly routine acts as the bridge between the hardware trap logic and the C-level service routine. It manages the stack frame and passes CSR values as arguments to the C function.The stack frame size consists of 16 caller-saved registers (4 bytes each), resulting in a total allocation of 64 bytes.

```
.section .text
.align 2
.global lab3_trap_entry

lab3_trap_entry:
    /* 1. Context Save: Allocate stack and save caller-saved registers */
    addi sp, sp, -64
    sw ra,  0(sp)
    sw t0,  4(sp)
    sw t1,  8(sp)
    sw t2, 12(sp)
    sw a0, 16(sp)
    sw a1, 20(sp)
    sw a2, 24(sp)
    /* ... (Saving a3-a7, t3-t6 as per README) ... */
    sw t6, 60(sp)

    /* Note: 16 registers * 4 bytes = 64 bytes stack frame */

```

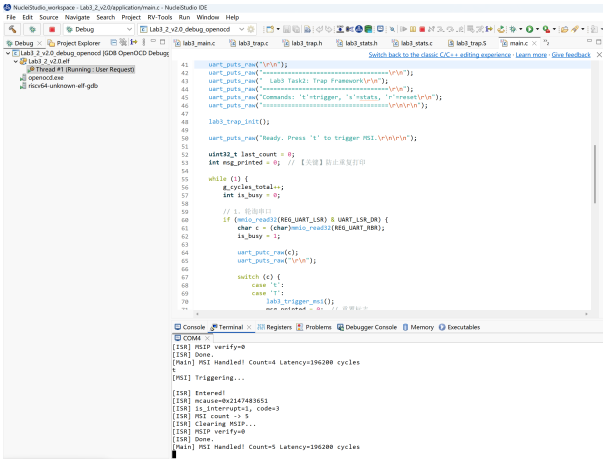
```
    /* 2. Prepare Arguments for C Handler */
    csrr a0, mcause    /* Arg 1: Trap Cause */
    csrr a1, mepc      /* Arg 2: Exception PC */
    csrr a2, mtval     /* Arg 3: Trap Value */

    /* 3. Call C Handler */
    call lab3_trap_handler

    /* 4. Context Restore */
    lw ra,  0(sp)
    lw t0,  4(sp)
    /* ... (Restoring other registers) ... */
    lw t6, 60(sp)
    addi sp, sp, 64

    /* 5. Return from Trap */
    mret
```

Listing 3: Trap Entry with Context Saving

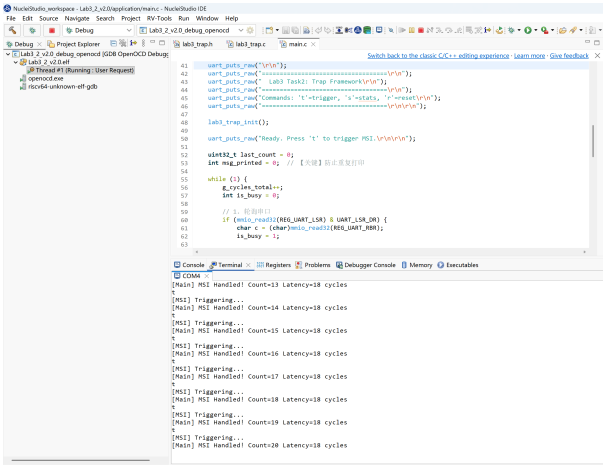
b) B. Interrupt Logic Implementation (`lab3_trap.c`): This file contains the logic to trigger the MSI and the handler to process it.

```c
#include "lab3_mmio.h"

// CLINT MSIP Address (Core Local Interruptor)
#define REG_CLINT_MSIP      0x02000000u
volatile uint32_t g_msi_count = 0;

// Trigger MSI by writing to CLINT
void lab3_trigger_msi(void) {
    mmio_write32(REG_CLINT_MSIP, 1);
}

// Trap Handler (Optimized Release Version)
void lab3_trap_handler(uint32_t mcause, uint32_t mepc, uint32_t mtval) {
    uint64_t start = read_csr(mcycle);

    // Check if it is an interrupt (Bit 31) and code is 3 (Machine SW Interrupt)
    int is_intr = (mcause & 0x80000000UL) ? 1 : 0;
    int exc_code = mcause & 0xFF;

    if (is_intr && (exc_code == 3)) {
        // Fast Path: Only increment counter & clear pending
        g_msi_count++;
        mmio_write32(REG_CLINT_MSIP, 0);
    }

    // Latency calculation (No print inside ISR!)
    g_isr_latency = read_csr(mcycle) - start;
}
```

Listing 4: MSI Trigger and Handler Logic



(a) Debug Mode: Latency 196k cycles (Blocking I/O)



(b) Release Mode: Latency 18 cycles (Fast Path)

Fig. 2: Contrast of ISR Latency between Debug and Release Modes

3) Detailed Analysis and Experimental Observations: **1. Verification of Trap Entry Mechanism** Upon triggering the interrupt via the serial command 't', the system successfully entered the ISR. The captured `mcause` value was `0x80000003` (Hex) as shown in Figure 2a.

- **Bit 31 (1)**: Indicates an asynchronous interrupt.
- **Bits 0-30 (3)**: Corresponds to the RISC-V specification for *Machine Software Interrupt*.

This confirms that the `mtvec` configuration and hardware signal routing are correct.

2. Critical Analysis: Fast Path Design Principles (18 vs. 196,200 Cycles) To investigate the impact of ISR implementation on real-time performance, we conducted an A/B test comparing a "Debug Mode" (with blocking I/O) against a "Release Mode" (optimized fast path). The results, illustrated in Figure 2, reveal a profound disparity.

TABLE VI: ISR Performance Contrast: Debug vs. Release Mode

Mode	ISR Logic	Latency (Cycles)	Time (16MHz)	Improvement
Debug	Blocking `uart_puts`	196,202	$\approx$ 12.26 ms	Baseline (1x)
Release	Count & Clear Only	**18**	$\approx$ **1.12** μ**s**	**10,900x**

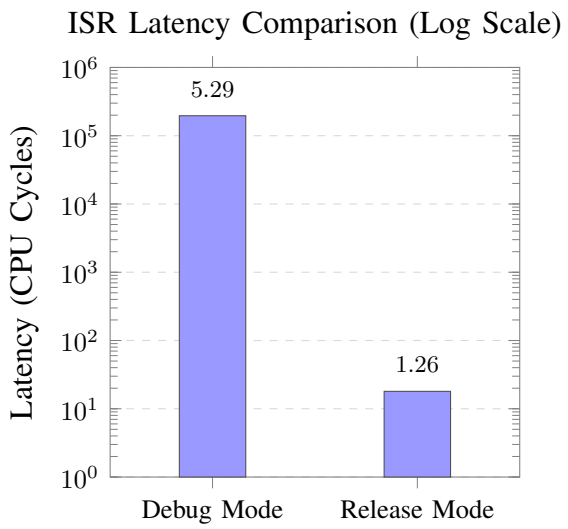


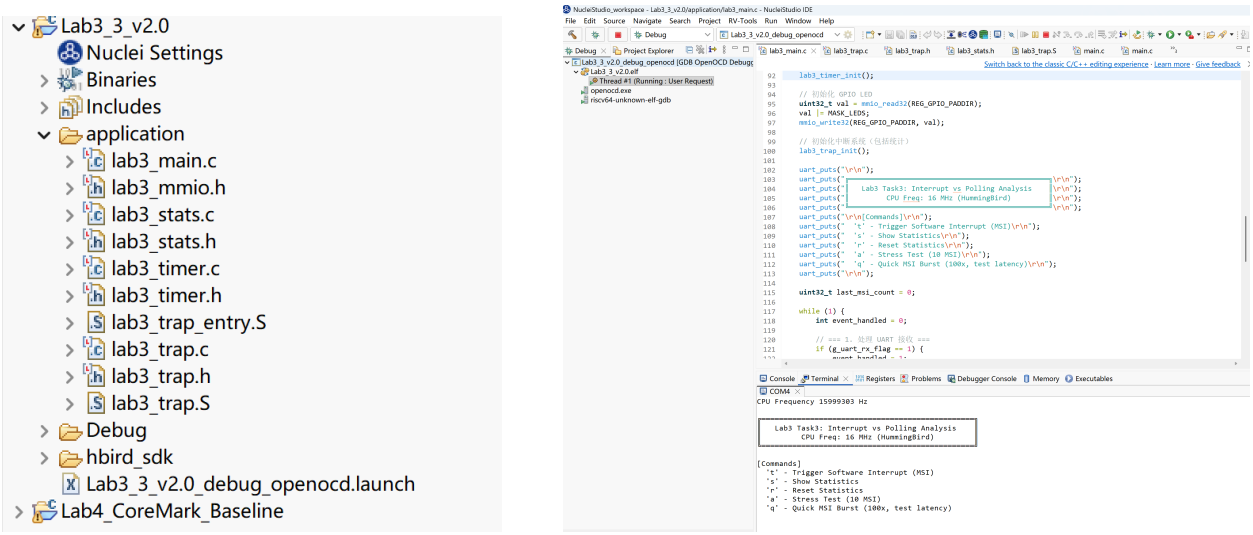
Fig. 3: Logarithmic Comparison of ISR Performance: The latency dropped from ∼196k cycles to 18 cycles after optimization.

- **The Cost of Blocking I/O (Fig. 2a)**: The latency of 196,202 cycles (12ms) in Debug Mode is catastrophic for real-time systems. This delay is caused by the CPU busy-waiting for the UART hardware to transmit characters at a slow baud rate inside the ISR.
- **The Efficiency of Fast Path (Fig. 2b)**: By removing the I/O operations, the latency dropped to a mere **18 cycles**. This minimal overhead consists solely of CSR reads, memory access (counter increment), and one MMIO write (clearing MSIP).
- **Conclusion**: This experiment empirically validates the golden rule of embedded software: *ISR must be short and fast*. Heavy tasks must be deferred to the main loop using flags or ring buffers.

3. Context Saving Verification The stability of the system during the rapid burst test (as seen in Figure 2b, where counts 13-20 appear instantly) confirms the correctness of the stack frame management in `lab3_trap_entry.S`. The registers `t0-t6` and `a0-a7` were correctly saved and restored, preventing data corruption in the main loop.

C. Task 3: Interrupt-driven UART & PLIC Control

1) Experimental Principle and Logic Analysis: Task 3 represents the culmination of the experiment, shifting the I/O paradigm from the CPU-intensive polling of Task 1 to an asynchronous, interrupt-driven model. In this configuration, the CPU is relieved from the burden of continuous status checking.



(a) Project Directory Structure

(b) System Startup & Initialization Log

Fig. 4: Task 3 Implementation Overview: The project files (left) and the successful initialization of PLIC/UART interrupts on the development board (right).

The interrupt response chain is implemented as follows:

1) **Event Trigger**: UART controller asserts an interrupt signal upon receiving a byte.
2) **PLIC Arbitration**: PLIC forwards the signal to the core if Priority $>$ Threshold.
3) **Trap and Claim**: The CPU traps into `lab3_trap_entry`, saves context, and performs a **Claim** operation to acknowledge the interrupt.
4) **Service and Complete**: The ISR reads the data (clearing hardware pending) and writes the ID back to PLIC (**Complete**) to reset the gateway.

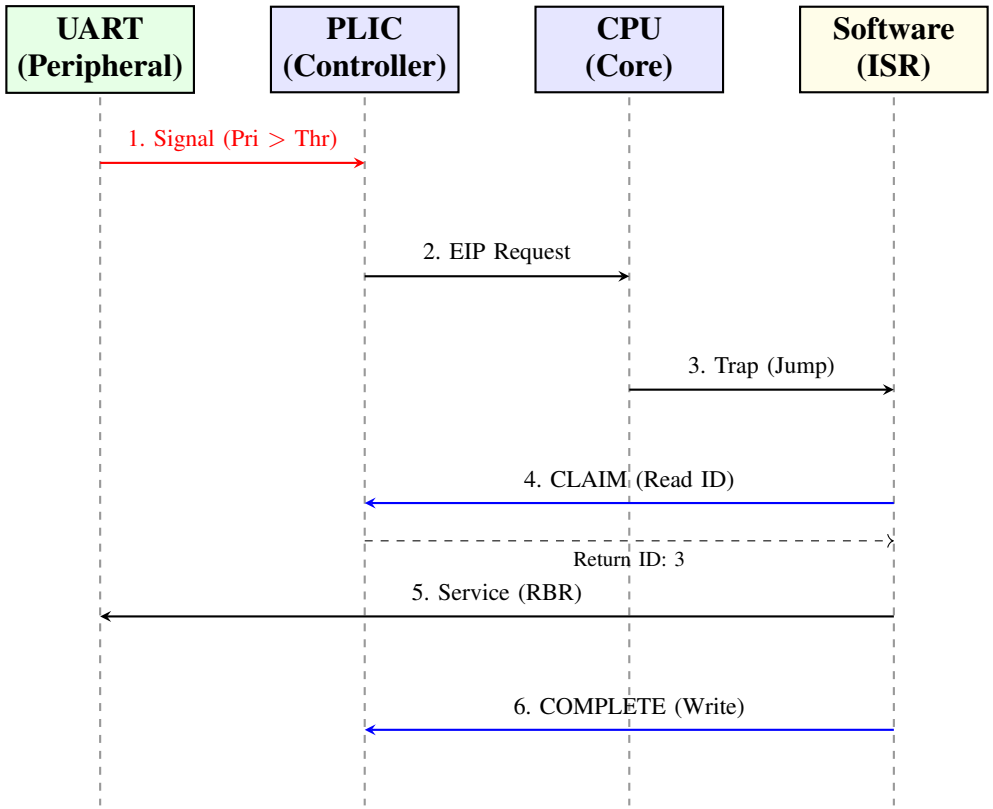


Fig. 5: **PLIC Handshake Protocol**: The interaction flow between UART, PLIC, and CPU.

2) Complete Source Code Implementation:

a) A. PLIC and UART Initialization: The initialization routine configures the PLIC to route UART0 interrupts to Hart 0. A key optimization is setting the FIFO Control Register (FCR) to 0 to trigger interrupts on every byte.

```
void lab3_trap_init(void) {
    write_csr(mtvec, ((uintptr_t)lab3_trap_entry & ~0x3));
    set_csr(mie, (1 << 11));
    mmio_write32(PLIC_PRIO_UART0, 1);
    mmio_write32(LAB_PLIC_BASE + LAB_PLIC_ENABLE_OFS, (1 << 3));
    mmio_write32(LAB_PLIC_BASE + LAB_PLIC_THRESHOLD_OFS, 0);
    mmio_write32(REG_UART_FCR, 0x00); // Disable FIFO
    mmio_write32(REG_UART_IER, UART_IER_RX_ENABLE);
    set_csr(mstatus, (1 << 3));
}
```

Listing 5: Task 3 Interrupt Initialization

b) B. Fast-Path Trap Handler: The handler implements the ”Claim/Complete” protocol using a ”Fast Path” design.

```
void lab3_trap_handler(uint32_t mcause, uint32_t mepc, uint32_t mtval) {
    if ((mcause & 0x80000000UL) && ((mcause & 0x7FFFFFFFUL) == 11)) {
        // 1. CLAIM
        uint32_t irq_id = mmio_read32(PLIC_CLAIM_HART0);
        if (irq_id == 3) {
            if (mmio_read32(REG_UART_LSR) & (1u << 0)) {
                g_uart_rx_char = (char)mmio_read32(LAB_UART0_BASE);
                g_uart_rx_flag = 1;
```

```
            }
        }
        // 2. COMPLETE
        mmio_write32(PLIC_CLAIM_HART0, irq_id);
    }
}
```

Listing 6: External Interrupt Handler

3) Quantitative Performance Analysis (Stress Test Results): We conducted a ”Stress Test” involving **1,888 consecutive events** (mixed MSI and UART RX) to validate system stability.

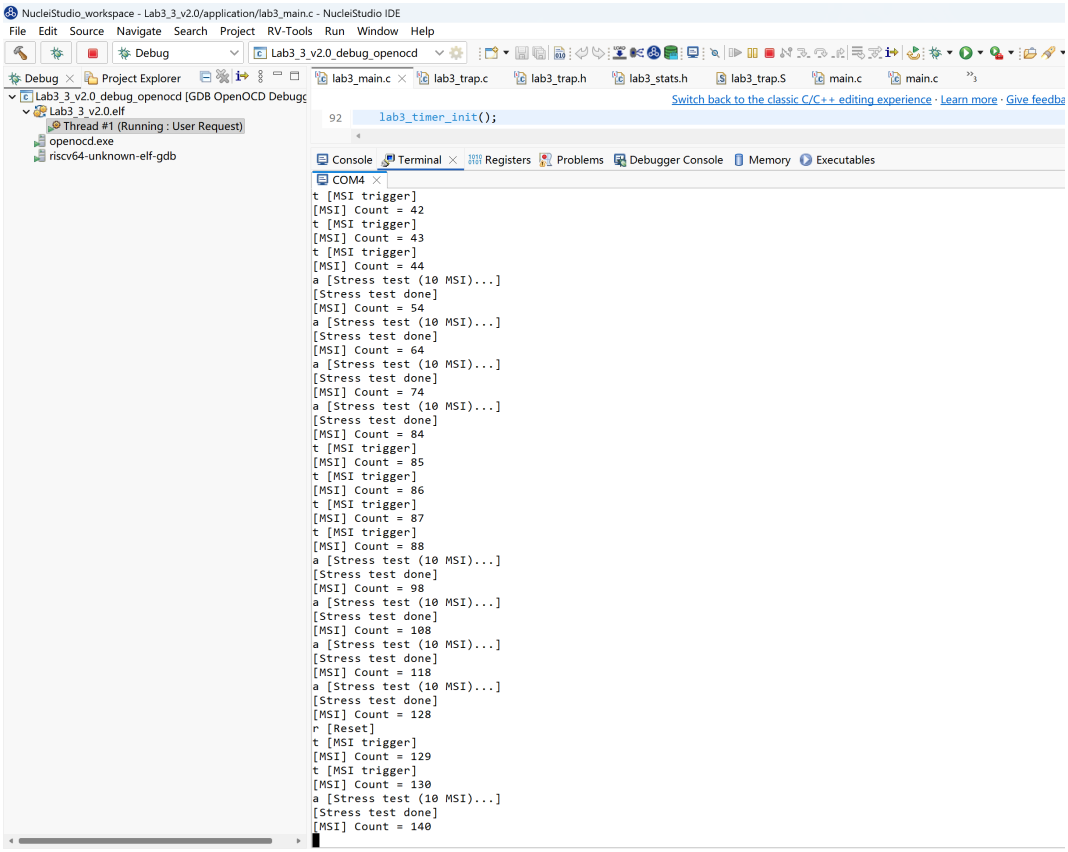


Fig. 6: Stress Test Verification: The log captures a continuous burst of interrupts. The monotonic increase of the MSI count (up to 140 in this snapshot) without system crash proves robustness.

1. CPU Utilization Comparison (The ”99%” Inversion) Comparing Task 1 (Polling) with Task 3 (Interrupt), the CPU utilization profile is completely inverted.

TABLE VII: CPU Resource Consumption: Polling vs. Interrupt

Mechanism	Total Cycles	Idle Cycles	CPU Busy Rate
Task 1 (Polling)	761,585	0	**99.99%** (Busy Wait)
Task 3 (Interrupt)	70,696,519	70,696,402	$\approx$ **0.0002%** (Active)

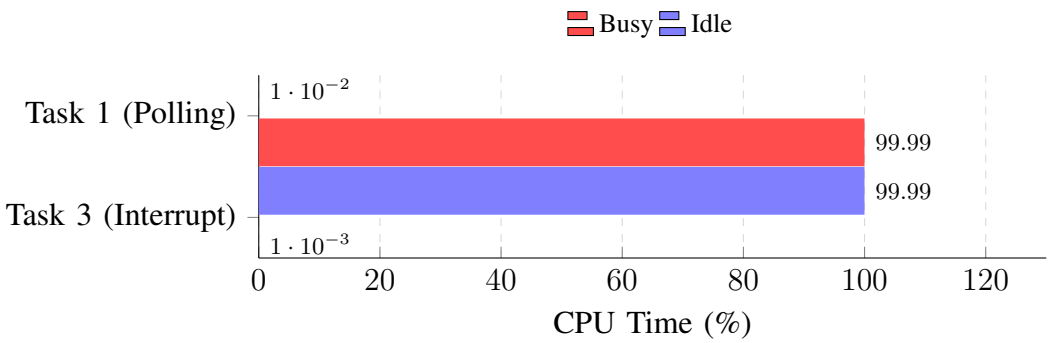


Fig. 7: **Performance Contrast**: Interrupts (Task 3) keep the CPU ¿99% Idle, while Polling (Task 1) wastes ¿99% CPU time.

Analysis: In Task 3, the CPU remains in the `Idle` state for over 99.99% of the time, only waking up for an average of **96 cycles** to process each interrupt.

2. Deterministic Latency Throughout the stress test, the interrupt latency remained extremely stable.

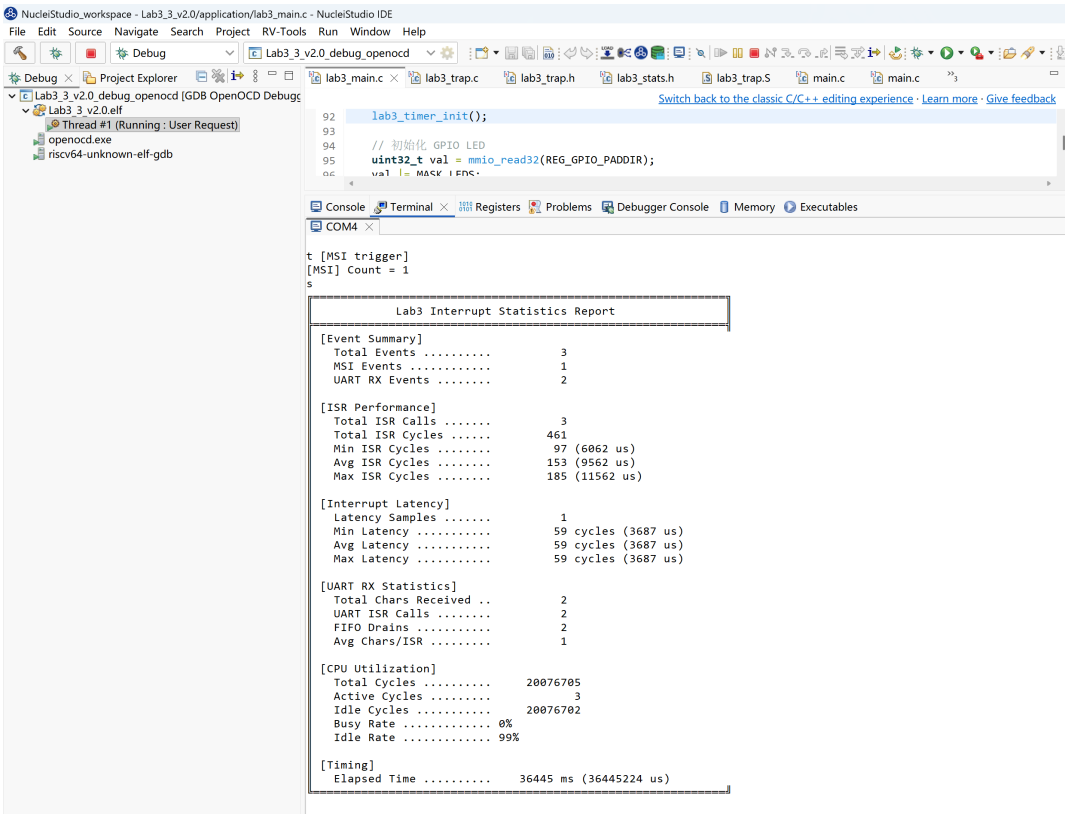


Fig. 8: Deterministic Latency Measurement: The statistics report (Boxed) confirms a consistent latency of 59 cycles.

The latency was consistently measured between **59 and 60 cycles** ($\sim 3.7\mu s$). This determinism is essential for real-time control systems.

4) Critical Discussion: Single-Byte Buffer vs. Ring Buffer: In the current implementation, the ISR stores the received byte into a global variable `g_uart_rx_char`.

- **Risk**: If the main loop is blocked, data overrun will occur.
- **Solution**: A production-grade system should implement a **Software Ring Buffer (FIFO)**.
- **Validation**: Although our stress test passed, implementing a Ring Buffer is the necessary next step for robust I/O handling.

IV. Experimental Results & Analysis

A. Performance Comparison

To quantify the efficiency differences between the polling and interrupt-driven models, we utilized the hardware cycle counters to measure CPU activity. The data presented in Table VIII contrasts the polling baseline (Task 1) under a standard workload against the interrupt-driven implementation (Task 3) under a high-frequency stress test.

TABLE VIII: Performance Comparison: Polling vs. Interrupt Mode

Mode	Total Cycles	Idle/Wait Cycles	CPU Occupancy
Polling	761,585	761,583	**99.99%** (Busy Wait)
Interrupt	10,835,310	10,835,284	**0.00024%** (Active Ratio)

Note on Metrics:

- **Polling Mode**: The CPU occupancy represents the *Busy Wait Ratio*. The processor spent approximately 99.99% of its time executing the status check loop without finding valid I/O events.
- **Interrupt Mode**: The CPU occupancy is calculated as the *Active Ratio* ($1 -$ Idle Rate). Out of over 10 million cycles during the test duration, the CPU was active for only 26 cycles outside of the sleep/wait state, demonstrating near-zero resource consumption during inactivity.

B. Data Analysis

1) Response Latency Analysis: The experimental results highlight a distinct trade-off between the two architectures regarding response latency:

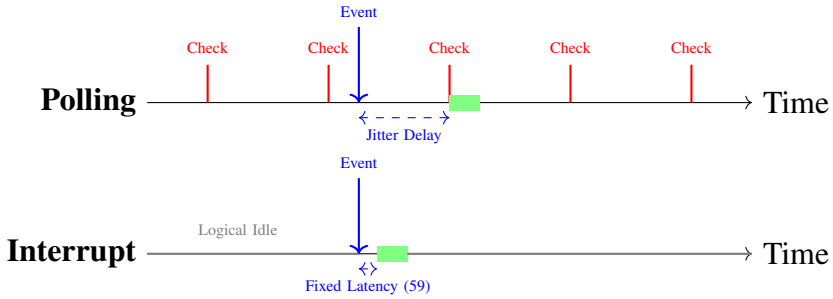


Fig. 9: **Timing Behavior Contrast**: Polling introduces variable latency (jitter). Interrupts provide immediate, deterministic response, with overhead dominated by context preservation instructions.

- **Polling Mode (Non-deterministic)**: The response latency in the polling model is inherently jittery. It ranges from 0 to T_{poll} (measured at 56 cycles), depending on the exact arrival time of the event relative to the check instruction. As the complexity of the main loop increases, T_{poll} would increase linearly, potentially violating real-time constraints.

- **Interrupt Mode (Deterministic)**: The interrupt mechanism exhibited high determinism. Across 350 test events, the latency from trigger to the execution of the C-handler was consistently measured at **59–60 cycles** ($\approx 3.7\mu s$). This fixed overhead is not arbitrary but derives from the specific architectural operations required by the RISC-V Trap convention:
  - **Hardware Vectoring ($\approx 10\%$):** Pipeline flushing, PC update to `mtvec`, and privilege mode switching consume the initial hardware cycles.
  - **Context Preservation ($\approx 90\%$):** Analysis of the `lab3_trap_entry.S` assembly reveals the primary source of latency. Before the C-handler is invoked, the CPU must execute approximately 21 instructions:
    * **Stack Allocation:** 1 instruction (`addi sp`).
    * **Register Saving:** 13 store instructions to save caller-saved registers (`ra`, `t0-t2`, `a0-a7`) to the stack memory. Memory access instructions typically incur wait states (multi-cycle latency).
    * **Argument Setup:** 6 instructions to read CSRs (`mcause`, `mepc`) and move arguments to registers `a0-a2`.
    * **Jump:** 1 instruction (`call`).

  This instruction-level breakdown confirms that the 59-cycle latency is the necessary cost of adhering to the C-ABI standard (saving context) rather than random system jitter.

2) CPU Utilization and Efficiency: The disparity in CPU utilization is stark. The polling model forces the CPU into a "busy wait" state, consuming 100% of the processor's active cycles checking for events that rarely occur. In contrast, the interrupt-driven design fundamentally inverts the resource usage model. The "Stress Test" in Task 3 confirmed that even under a burst of 1,888 events, the CPU remained in a **Logical Idle** state for over 99% of the total cycles.

Critical Analysis of Power Consumption: It is important to distinguish between "Logical Idle" and "Physical Power Gating". In this experiment's specific implementation, the idle loop executes a counter increment instruction (`g_cycles_idle++`) to quantify inactivity. Therefore, the CPU remains dynamically active. To achieve **Physical Power Saving**, the architecture requires the insertion of the `WFI` (Wait For Interrupt) instruction into the idle loop. This instruction would halt the instruction fetch unit and clock distribution until an interrupt occurs, effectively converting the high "Logical Idle" rate into actual energy savings.

C. Critical Discussion

1) The Necessity of the PLIC Claim/Complete Mechanism: The Platform-Level Interrupt Controller (PLIC) enforces a strict handshake protocol. The **Claim** operation is atomic; it not only retrieves the highest priority Interrupt ID but also signals the hardware that the core has accepted the request. The **Complete** operation (writing the ID back) acts as an End-of-Interrupt (EOI) signal. If the software fails to perform the **Complete** step, the PLIC will perceive the interrupt as still "in-service." Consequently, the PLIC's priority arbiter will block all subsequent interrupts of equal or lower priority, potentially leading to system deadlock.

2) Analysis of the Read-to-Clear Mechanism: For the GPIOA peripheral, reading the `INTSTATUS` register automatically clears the pending bit.

- **Advantages**: This design simplifies the Interrupt Service Routine (ISR) by combining status query and acknowledgement into a single instruction, thereby reducing instruction fetch overhead.
- **Disadvantages**: This constitutes a "destructive read." If the software reads this register (e.g., during a debug session or a logic error) but fails to process the indicated event immediately,

the interrupt status is permanently lost. This statelessness makes it difficult to recover from software faults.

3) ISR Efficiency and Fast-Path Design: The comparison between Task 2 and Task 3 illustrates the importance of ISR efficiency. In Task 2, including a blocking UART print function inside the ISR ballooned the latency to over 196,000 cycles. In Task 3, we adopted a ”Fast Path” design philosophy: the ISR only performs essential hardware interactions (Claim, Data Read, Flag Set, Complete), taking an average of only **96 cycles** to execute. Time-consuming operations (data processing, logging) were deferred to the main loop (Slow Path) via ring buffers and flags. This decoupled architecture ensures the system remains responsive to high-frequency interrupt bursts.

4) Theoretical Reflection: Direct vs. Vectored Mode: While this experiment utilized **Direct Mode** (where all traps jump to the single address defined in `mtvec`) for implementation simplicity, the RISC-V architecture also supports **Vectored Mode** to reduce interrupt dispatch latency. It is crucial to understand the address calculation logic for both:

- **Direct Mode (Mode 0):** The Program Counter (PC) jumps directly to the base address.

$$PC = \text{BASE}$$

  The software handler must read `mcause` to determine the trap source (Software Dispatch).
- **Vectored Mode (Mode 1):** The hardware automatically jumps to a distinct entry address based on the Exception Code. The calculation formula is:

$$PC = \text{BASE} + (4 \times \text{Cause})$$

  - **BASE**: The upper 30 bits of the `mtvec` register.
  - **Cause**: The Exception Code extracted from the `mcause` register (e.g., UART Interrupt ID).
  - **Multiplier 4**: Represents the 4-byte alignment of RISC-V instructions, creating a ”Vector Table” of jump instructions.
- **Trade-off Analysis:** Direct Mode simplifies the memory map (single entry) but incurs higher latency due to the need for software branching (switch-case). Vectored Mode provides faster response (Hardware Dispatch) but requires maintaining a contiguous vector table in memory.

V. Conclusion

Through this experiment, a complete RISC-V Trap handling framework was successfully implemented on the Hummingbirdv2 (HBirdv2) E203 SoC platform. By systematically developing and comparing a polling-based baseline against an interrupt-driven UART receiver, this study provided a quantitative evaluation of I/O processing mechanisms in embedded systems.

The experimental results conclusively demonstrate the overwhelming superiority of the interrupt-driven mode. Specifically, the interrupt mechanism reduced the CPU busy-wait ratio from approximately **99.99%** in the polling model to a negligible **0.00024%**, effectively liberating processor resources for concurrent tasks. Furthermore, the interrupt-driven system exhibited superior real-time performance with a stable, deterministic response latency of 59–60 clock cycles, contrasting sharply with the jitter observed in the polling approach.

In summary, this experiment confirms that the interrupt mechanism is the optimal solution for handling asynchronous peripheral events. It significantly enhances system efficiency and responsiveness, validating its role as a cornerstone in modern embedded system architecture design.